

Interrupts in Embedded Systems

Concepts, categories, hardware, external interrupts, and ATmega32A registers



Session Topics

- What is an interrupt?
- Maskable vs Non-maskable interrupts
- Hardware vs Software interrupts
- Polling vs Interrupt-driven operation
- Interrupt vector tables
- Internal vs External interrupts
- ATmega32A external interrupts and registers



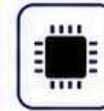
What is an Interrupt?

An event that pauses normal execution to service a request.



Categories

Maskable vs Non-maskable
Hardware vs Software



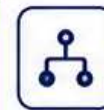
Hardware & Sources

Internal and external interrupt sources in ATmega32A.



Vector Tables

Each interrupt has a dedicated vector address.



ATmega32A Registers

External interrupts INT0/INT1 and related control registers.

What Is an Interrupt?

An event that temporarily pauses the main program to service an urgent request.

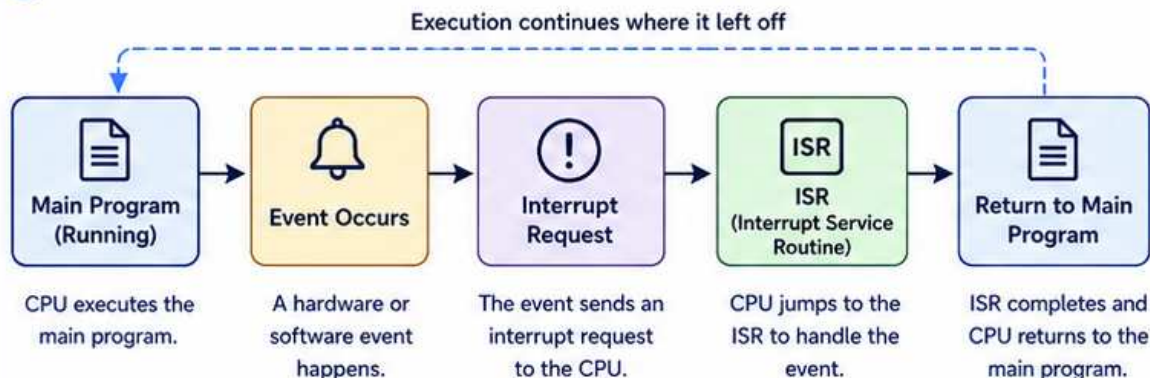
1 Definition



An interrupt is a signal that notifies the CPU that an important event has occurred and needs immediate attention.

The CPU pauses the current code, services the event, then returns to where it left off.

2 How an Interrupt Works



3 Examples of Events



Button Press
User presses a button.



Timer Overflow
A timer reaches its limit.



UART Receive
A byte is received on a serial port.

4 Why Use Interrupts?

- ✓ Fast response to important events
- ✓ Efficient CPU usage (no busy waiting)
- ✓ Handles multiple events asynchronously
- ✓ Enables real-time and concurrent operation

5 Interrupt vs Function

Function Call		Interrupt	
	Called explicitly by software (e.g., <code>my_function()</code>).		Triggered by hardware or an event automatically.
	The caller decides when and where it runs.		The CPU transfers control automatically to the ISR.



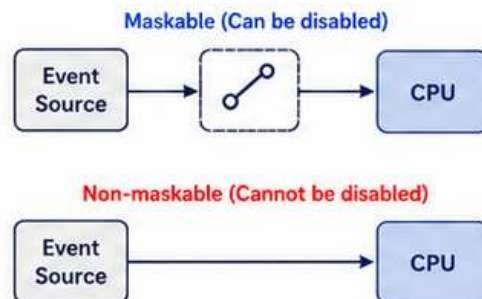
Interrupt Categories

Main classifications used in embedded systems.

1 Maskable vs Non-maskable interrupts

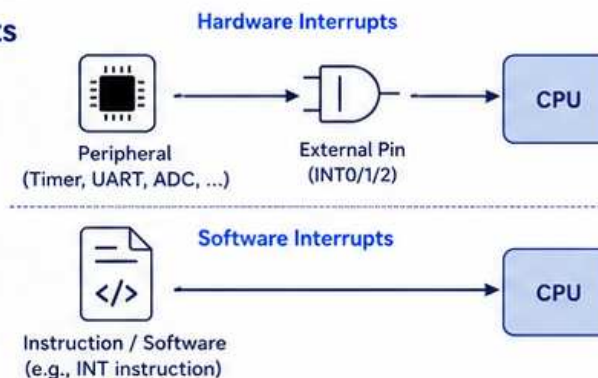
- Maskable interrupts can be enabled or disabled by software (global or individual enable bits).
- Non-maskable interrupts cannot be ignored and are reserved for critical events.

i ATmega32A mainly uses maskable interrupts. Reset and fault-type events are examples of non-maskable concepts.



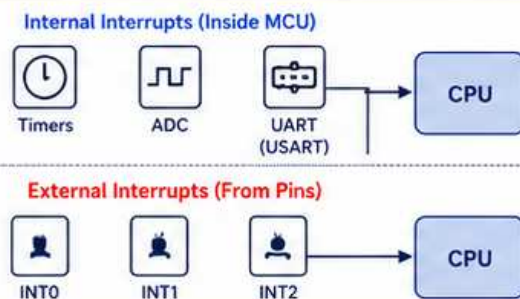
2 Hardware vs Software interrupts

- Hardware interrupts are generated by peripherals or external pins.
- Software interrupts are generated by instructions or software mechanisms.



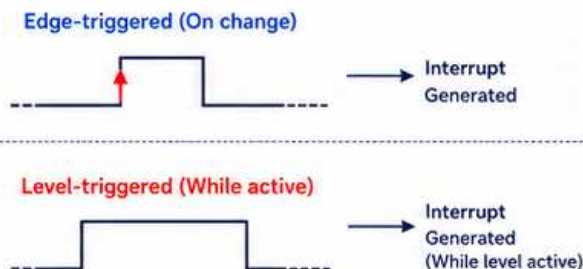
3 Internal vs External interrupts

- Internal interrupts originate inside the MCU from on-chip peripherals and modules.
- External interrupts come from external pins connected to outside sources.



4 Synchronous note

- Interrupts are event-driven.
- They can be edge-triggered or level-triggered depending on the source and configuration.



Summary

Interrupts are classified in multiple ways to describe how they are generated and handled.

ATmega32A primarily uses maskable, hardware, internal, event-driven interrupts (edge or level as configured).

Polling vs Interrupts

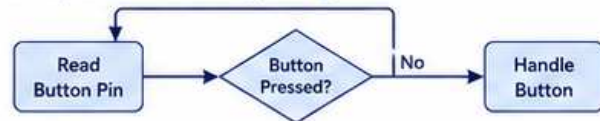
Two ways to detect and service events.

1 Polling

CPU continuously checks a flag or pin.

-  The CPU repeatedly reads the input to see if an event occurred.
-  Simple to implement and understand.
-  Wastes CPU time and may respond late (depends on polling rate).

Example: Button (using polling)



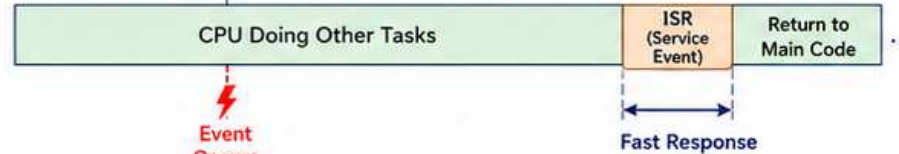
- CPU keeps reading the button pin until pressed.
- If polling is slow, response may be delayed.

Time

Polling Mode



Interrupt Mode



2 Interrupts

CPU services the event only when it occurs.

-  An event triggers an interrupt. The CPU jumps to the ISR.
-  CPU is free to do other tasks until the event happens.
-  Faster response and more efficient CPU usage.

Example: Button (using interrupt)



- CPU continues other work until button is pressed.
- Interrupt immediately triggers the ISR.

Best use cases

Use Polling when:

- Events are infrequent and not time-critical
- Simplicity is more important than efficiency
- System has no interrupt capability

Use Interrupts when:

- Fast response is needed
- Events are irregular or time-critical
- CPU resources must be used efficiently



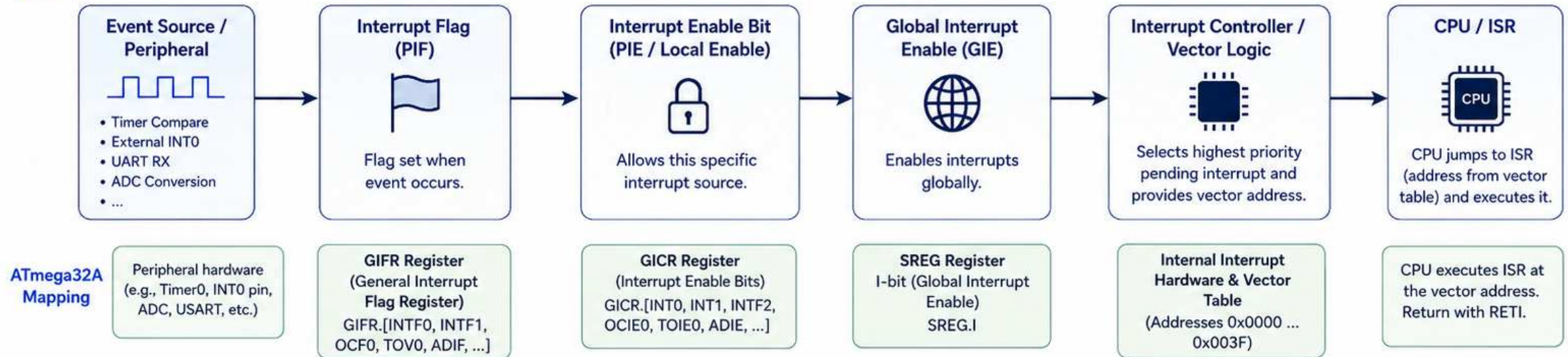
Key difference

Polling: CPU asks "Did an event happen?" many times.

Interrupts: The event tells the CPU "An event happened!"

Interrupt Hardware Logic

How an interrupt request reaches the CPU.



What each term means



PIF (Interrupt Flag)

- Set by hardware when the event occurs.
- "This happened."



PIE (Interrupt Enable)

- When set, this specific interrupt source is allowed to generate a request.
- "This source is allowed."



GIE (Global Interrupt Enable)

- The I-bit in SREG.
- When 1, the CPU will respond to enabled interrupt requests.
- "Interrupts are enabled globally."



ISR (Interrupt Service Routine)

- The function that handles the interrupt.
- Executed by the CPU after vectoring to the ISR address.

★ Note

Some systems use a Programmable Interrupt Controller (PIC) to handle multiple interrupt sources with priorities and vectoring.

Interrupt Vector Table

Where the processor jumps when an interrupt happens.



The **interrupt vector table** is a **table of addresses**.

Each entry points to the **service routine (ISR)** the CPU jumps to when that interrupt occurs.

Vector No.	Source (Interrupt)	Vector Address (Example)
0	RESET	0x0000
1	INT0 – External Interrupt 0	0x0002
2	INT1 – External Interrupt 1	0x0004
3	INT2 – External Interrupt 2	0x0006
16	TIMER0 Overflow	0x0020
24	USART, Rx Complete	0x0030

i Addresses are examples. Actual addresses depend on the device and architecture.

Static vs. Dynamic Vector Tables



Static (Fixed) Vector Table

- Vector table is located at fixed addresses defined by the hardware.
- Each interrupt has a fixed entry.
- Simple and predictable.



Dynamic (Relocatable) Vector Table

- The base address of the vector table can be changed (remapped).
- Some processors allow modifying vector entries or relocating the table to RAM/Flash.
- Provides flexibility for bootloaders, applications, and advanced use cases.



Note: Classic AVR devices such as ATmega32A primarily use fixed (static) vectors.

Internal and External Interrupts

Interrupt sources inside and outside the MCU.

1 Internal Interrupts (Inside the MCU)

Generated by peripherals and events within the ATmega32A.



Timer Overflow

Triggered when a timer/counter overflows.



Compare Match

Triggered when timer value matches a compare register.



ADC Conversion Complete

Triggered when an ADC conversion finishes.



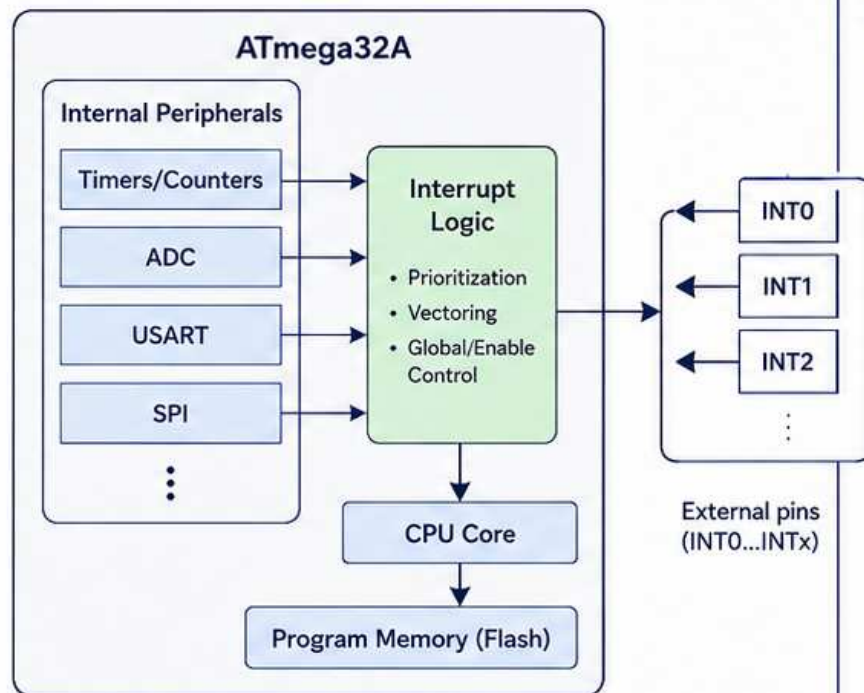
USART Receive Complete

Triggered when a byte is received in the USART.



SPI Transfer Complete

Triggered when an SPI transfer completes.



2 External Interrupts (Outside the MCU)

Generated by signals coming from outside the ATmega32A.



Push Buttons

User buttons connected to external interrupt pins.



Sensors

Motion, proximity, temperature, or other sensors.



Encoders

Incremental encoders generating index or direction signals.



Emergency Lines

Fault, alarm, or emergency signals that require immediate attention.



Note: This session will focus especially on **external interrupts (INT0 and INT1)**, their configuration, and how to use them effectively.

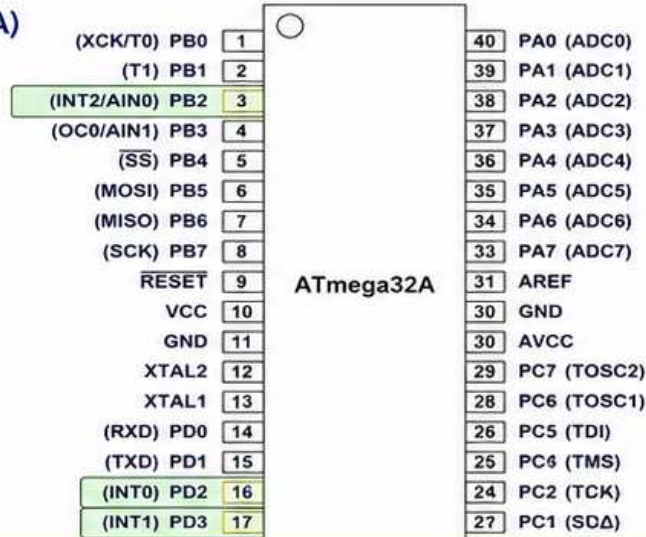
External Interrupts in ATmega32A

INT0, INT1, and INT2 pins and their trigger modes.

External Interrupt Pins (ATmega32A)

- INT0 = PD2
- INT1 = PD3
- INT2 = PB2

These pins can generate an interrupt request to the CPU when an external event occurs.



Interrupt Trigger Modes

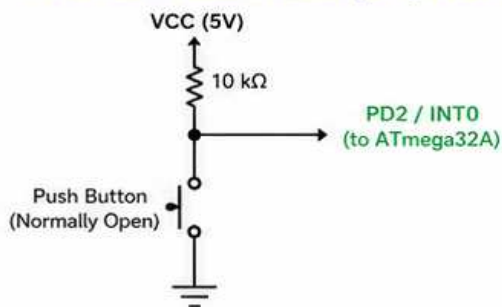
Interrupt Pin	Trigger Modes	How to Select
INT0 (PD2) INT1 (PD3)	• Low level • Any logical change • Falling edge • Rising edge	Configured using ISC01:ISC00 (for INT0) and ISC11:ISC10 (for INT1) in MCUCR.
INT2 (PB2)	• Falling edge (default) • Rising edge	Selected by ISC2 bit in MCUCSR.



INT0 and INT1 support four trigger modes.

INT2 supports **edge-triggered interrupt only** (configured by ISC2).

INT0 Push-Button Example (Active Low)



- The pin is normally HIGH via the pull-up resistor.
- Pressing the button pulls INT0 (PD2) LOW.
- **Configure INT0 for falling edge interrupt** (common for buttons and sensors).
- Edge interrupts are commonly used for buttons and sensors because they are fast and reliable.
- Low-level mode can be useful for signals that remain active for a period of time (e.g., alarm, window open, level detection).

Key Notes



External interrupts allow the MCU to respond immediately to important events without continuous polling.



Use edge-triggered mode for momentary events (buttons, sensors, encoders).



Use low-level mode when the source stays active until cleared.



Remember to enable the interrupt (GICR) and set the corresponding ISR (INT0_vect, INT1_vect, INT2_vect).

ATmega32A External Interrupt Registers

Global enable and control registers from the datasheet.

Register	Name / Bits	Purpose
SREG	I (bit 7) Global Interrupt Enable	The I-bit globally enables (1) or disables (0) all interrupts.
GICR	INT0 (bit 6), INT1 (bit 7), INT2 (bit 5) Interrupt Enable Bits	Enables external interrupts INT0, INT1, and INT2.
GIFR	INTF0 (bit 6), INTF1 (bit 7), INTF2 (bit 5) Interrupt Flag Bits	Flags set when an external interrupt occurs. Cleared by hardware (upon ISR) or by writing a logic 1.
MCUCR	ISC01:0 (bits 1:0) – INT0 ISC11:10 (bits 3:2) – INT1 Interrupt Sense Control	Selects trigger mode for INT0 and INT1 using ISC00, ISC01, ISC10, and ISC11.
MCUCSR	ISC2 (bit 6) Interrupt Sense Control 2	Selects trigger mode for INT2 using ISC2.



External Interrupt Vectors

Interrupt	Vector Name	Description
INT0	ISR(INT0_vect)	External Interrupt 0
INT1	ISR(INT1_vect)	External Interrupt 1
INT2	ISR(INT2_vect)	External Interrupt 2



Typical Setup Sequence

- 1 Configure the external interrupt pin (INT0/1/2) as input.
- 2 Set the trigger mode (ISC bits in MCUCR / MCUCSR).
- 3 Enable the desired interrupt in GICR (INT0 / INT1 / INT2).
- 4 Enable global interrupts: set I-bit in SREG (use SEI instruction).
- 5 Write the Interrupt Service Routine: ISR(INTx_vect).

Interrupt Session Summary

Main ideas from this lecture

1 What is an Interrupt?



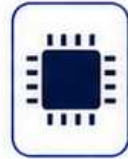
An interrupt is an event that pauses normal program execution and transfers control to an Interrupt Service Routine (ISR). After handling, execution returns to the main program.

2 Maskable vs Non-maskable



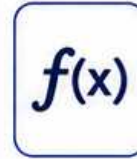
- **Maskable Interrupts:** Can be enabled or disabled by software.
- **Non-maskable Interrupts:** Cannot be disabled by software; always active.

3 Hardware vs Software Interrupts



- **Hardware Interrupts:** Generated by external or internal hardware events.
- **Software Interrupts:** Generated by executing an instruction (e.g., INT instruction).

4 Interrupt vs Function



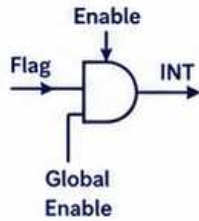
- **Function:** Called by the program; predictable and synchronous.
- **Interrupt:** Called by an event; asynchronous and time-critical.

5 Polling vs Interrupts



- **Polling:** CPU repeatedly checks status; wastes time and increases latency.
- **Interrupts:** Event-driven; CPU reacts immediately and uses time efficiently.

6 Interrupt Hardware Logic



- **Flag:** Set when an event occurs.
- **Enable:** Enables that source.
- **Global Enable (I-bit in SREG):** Enables all interrupts.
- **Interrupt occurs when:** Flag AND Enable AND Global Enable = 1.

7 Vector Tables & Addressing



- **Vector table** maps each interrupt to its ISR address.
- **Static Vectoring:** Fixed addresses (e.g., ATmega32A).
- **Dynamic Vectoring:** Address provided by device (e.g., some peripherals).

8 ATmega32A External Interrupts & Key Registers



- **External interrupts:** INT0, INT1.
- **Key Registers:**
 - SREG – Global interrupt enable (I-bit)
 - GICR – Interrupt enable control
 - GIFR – Interrupt flag register
 - MCUCR – Interrupt control (sense)
 - MCUCSR – MCU status control



Interrupts improve responsiveness and efficiency in embedded systems.

In ATmega32A, external interrupts are controlled mainly by SREG, GICR, GIFR, MCUCR, and MCUCSR.